DepGuard AI

Overall summary: AI-assisted dependency resilience and migration verification

Hacktoberfest 2026 · Track 1: Best Use of Gemma 4 · Status as of 15 October 2026 · Built inside the LinkWatch repository

1. What it is

DepGuard AI is a local tool for Python projects. It finds risky dependencies, asks a local **Gemma 4** model what to do about each one, proposes repairs, and runs the project's own tests in an isolated copy to check each repair. The evidence is recorded in a **Dependency Exit Capsule**. It runs as a dashboard at `http://127.0.0.1:8765/`.

> **Design rule: Gemma only advises.** Facts come from deterministic code, package-index metadata, or executed tests. Every item in the dashboard and the capsule is tagged by where it came from: Observed · Package metadata · Heuristic · Unknown · AI-generated · unverified · Verified by execution

2. The workflow (the dashboard's seven steps)

1 Repository > 2 Findings > 3 Ask Gemma 4 > 4 Repair > 5 Verify > 6 Publish > 7 Reports & capsule

1 · Repository	Open a **ZIP**, a **local Git repo** (branch, commit and dirty state recorded), or a **GitHub** repo (safe shallow clone, exact commit recorded). Untrusted sources need explicit confirmation before their tests run.
2 · Findings	A deterministic scan reads `requirements*.txt` and `pyproject.toml`; lockfiles and unsupported formats (Pipfile, poetry.lock and others) are detected and reported, not parsed. It maps imports with file:line, builds a dependency → module → test impact graph, and applies documented severity rules. Advisory, yanked, Requires-Python and license data come from the package index; that lookup is opt-in and cached with its source URL and fetch time.
3 · Ask Gemma 4	Every at-risk dependency gets a schema-validated answer: replace, upgrade, remove, keep, investigate, or "**no reliable recommendation**". Gemma is given the evidence and told not to contradict it.
4 · Repair	Each finding is *Repair*, *Ignore* or *Review later*. **Mode A** is user-triggered. **Mode B** only *prepares* fixes, with bounded retries that feed test failures back as data. Patches show per-file diffs with reason, origin, evidence and status; unsafe or stale patches are refused.
5 · Verify	The original and patched copies each run in an isolated virtual environment. Every step records its command, exit code, duration and timeout. Results are classified *verified*, *partially verified*, *failed*, *blocked* or *not run*, never optimistically.
6 · Publish	Export a `git apply`-compatible patch, then run pre-publish checks (secrets, hygiene, unintended changes). Optionally publish a **new** GitHub branch and pull request with two separately typed approvals. It never merges and never force-pushes.
7 · Reports & capsule	JSON, Markdown and HTML reports, plus one capsule per dependency, all schema-validated.

Also in the dashboard: dependency inventory with a heuristic health score, replacement comparison (evidence only; it never claims two packages are equivalent), scan history, quick scan, and experiments.

3. Who is allowed to say what

Source	Provides	Can it be wrong?
Deterministic code	manifests, imports, impact graph, severity	Static analysis only
Package index	advisories, yanked flags, Python and license metadata	Lists only what the index knows; offline means "not checked"
Gemma 4	recommendations and proposed edits	Yes, so everything it says is labelled unverified
Executed tests	whether a patch actually passes	Passing tests do not prove full compatibility

4. Safety model

- **Local only.** Inference runs through Ollama (`gemma4:e2b`); no paid or hosted APIs. The server binds `127.0.0.1` and rejects foreign `Host` / `Origin` headers and POSTs without `X-DepGuard`.
- **No command surface.** There is no shell or arbitrary-command endpoint; the 21 API routes only start the dashboard's own fixed jobs. Paths are contained, and traversal attempts return 404.
- **Nothing applied automatically.** Applying needs a typed `APPLY` and goes to a working copy only; the original tree is never touched by default.
- **Secrets.** GitHub tokens live in memory only and are redacted everywhere. The working tree scans clean for credential formats.
- **Untrusted text.** The prompts tell Gemma to treat repository text (files, comments, READMEs) as data, never as instructions. That is a safeguard, not a guarantee, so every edit it proposes is validated and then tested.

5. What it found on LinkWatch (real runs)

- 26 declared dependencies and **11 findings**: 4 critical, 2 medium, 5 low.
- Baseline: **9 passed, 1 failed**, which is the real Jinja2 2.11.3 `soft_unicode` break.
- Gemma proposed `Jinja2>=3.1.3` in both manifests. In isolation that gave **10 passed, 0 failed**. It was applied only to the working copy, and the original manifests are unchanged.
- An advisory-coverage check added afterwards shows that `>=3.1.3` still allows versions affected by advisories the index lists as fixed only in 3.1.4 to 3.1.6. It is unit-tested against the real advisory data and was not re-run live.
- A separate Jinja2-only experiment (baseline 9/1, proposed 10/0) is recorded in the demo capsule.
- Gemma's combined proposal in the demo run failed (`pkg_resources` missing, exit 4). That failure is recorded rather than hidden.

6. Verification status

Check	Result
Full test suite, Python 3.12.10	**366 passed**, 3 live tests deselected, exit 0
Full test suite, Python 3.13.14	**366 passed**, 3 live tests deselected, exit 0
Live Gemma 4 test (`gemma4:e2b`, local Ollama 0.40.4)	**1 passed** in 134 s, exit 0
LinkWatch's own suite, original tree	**9 passed, 1 failed** (known Jinja2 failure, left unfixed on purpose); pytest exit 1
Capsule schema validation	5 of 5 capsules valid
Web smoke test on the running server	20 of 20 checks passed
JavaScript syntax check	All 7 files OK
Secrets scan of the working tree	0 findings

7. Where the code lives

Analysis	`scanner.py`, `usage.py`, `impact.py`, `findings.py`, `health.py`, `metadata.py`, `compare.py`
Model	`advisor.py`
Repair & verify	`repair/` (patch, planner, state, advisory), `verify.py`
Checks & publish	`prepublish/` (secrets, hygiene, review), `git_integration/` (github, pull_request)
Inputs	`repo_input.py`, `acquire.py`
Output	`capsule.py`, `capsule.schema.json`, `report_html.py`
Dashboard	`web/` : stdlib server with plain HTML/CSS/JS. Dependencies: `jsonschema`, `packaging`, `pytest`.

All paths are inside the `depguard/` folder of the repository.

8. Honest limits

- **Isolation.** Verification uses a virtual environment plus a timeout. It is not a container and has no CPU or memory limits.
- **GitHub publishing.** The branch and pull-request flow has only been tested against a mock, never live.
- **Coverage.** There is no lockfile parsing. "Transitive" dependencies are only those observed installed in the test environment, not a resolved graph.
- **Advisories and maintenance.** Advisory data is whatever the package index lists; maintenance activity beyond release dates is never checked.
- **Model quality.** `gemma4:e2b` is small and can propose weak fixes, so every proposal must be verified.

9. Run it

```
cd H26_Elite_DepGuardAI
.venv\Scripts\python -m depguard.web                          # then open http://127.0.0.1:8765/
.venv\Scripts\python -m pytest depguard/tests -k "not live"    # 366 tests, about 2 minutes
```

Nothing has been committed or pushed; the git and GitHub steps are left to the repository owner.

Every figure in this document comes from commands that were actually run during development; none are estimated. Advisory data reflects what the public package index listed when it was fetched; it is not a security audit.